

Fundamentals of program development

Files and dictionaries

Chalmers



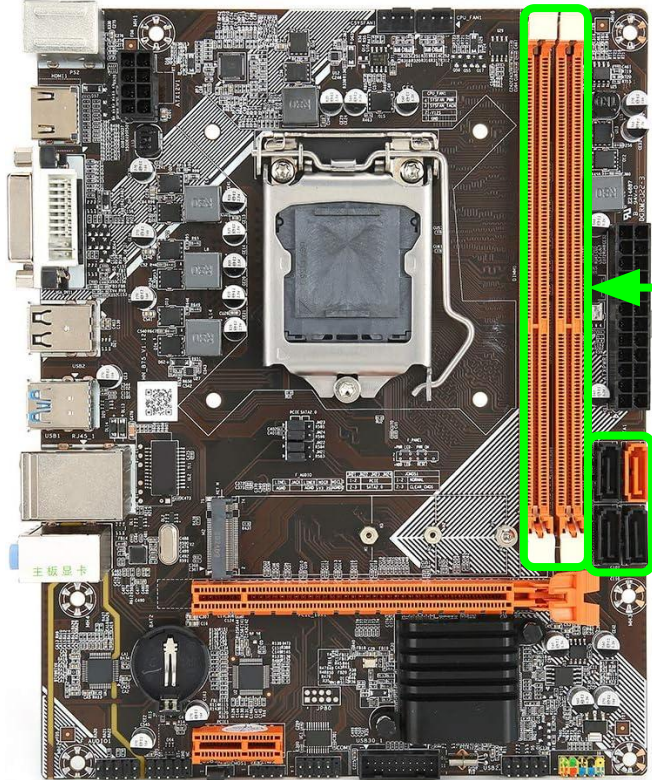
Agenda

Files

Dictionaries

Files

Primary and secondary memory



Primary. Fast access. Not a lot of it. Transient.

Secondary. Slower access. A lot of it. Persistent.



Files and directories

```
python-course
├── exam
│   ├── ht2023
│   │   ├── example
│   │   ├── facit.org
│   │   ├── facit.pdf
│   │   ├── q1.py
│   │   ├── q2.py
│   │   └── q3.py
│   ├── st22
│   │   ├── facit.org
│   │   ├── unk.py
│   │   └── vt23
│   │       ├── facit.org
│   │       └── q1.py
│   └── lecture10
│       ├── async.py
│       ├── cheat.py
│       ├── graphics.py
│       ├── racer_async.py
│       └── r.py
└── lecture11
    ├── cont.py
    └── dot.py
```

etc.

- Files are an abstraction of storage devices
- To access the contents of a file, they must first be loaded to RAM
 - Note that a file can be larger than the total RAM available
- There are typically a lot of files.
 - They are organised in directories
 - Directories can be nested
- See also the lecture about OS basics

Read a whole file as a string

```
>>> file = open("input.txt")

>>> content = file.read()

>>> file.close()

>>> print(content)
<.. content >
```

open(<filename>) creates an object which represents the file in Python.

<filename> is relative to the current directory

<file>.read() returns the content of the file as a string.

<file>.close() must be used at the end to close the file.

The content of the file is still available in the string

Read the whole file as a list of lines

```
>>> file = open("input.txt")

>>> lines = file.readlines()

>>> file.close()

>>> print(lines)
[<.. line 1 ..>, <.. line 2 ..>, ...]
```

open(<filename>) creates an object which represents the file in Python.

<file>.readlines() returns a list of all lines in the file. Each line is a string.

<file>.close() must be used at the end to close the file.

The content of the file is still available in the list

NOTE: even though your storage devices are likely to be random-access, the python interface to files is sequential by default.

Data in files

The easiest way: text files

- one **entry** per **line**
- **columns** which are **separated** with tabs, commas or semicolons

Alternatively: structured document

- XML
- JSON
- spreadsheet (Excel)
- relational database (SQL)
- binary format, often **compressed**

We will limit ourselves to semicolon (;)
separated text files (**.csv**) (comma and tab
are also usual)

```
attribut ; attribute ; N
metod ; method ; N
formatera ; format ; V
formatering ; formatting ; N
muterbar ; mutable ; A
icke-muterbar ; non-mutable ; A
skräp ; garbage ; N
referera ; refer ; V
muterbarhet ; mutability ; N
effektiv ; efficient ; A
ineffektiv ; inefficient ; A
optimering ; optimization ; N
element ; element ; N
öppna ; open ; V
stänga ; close ; V
trädstruktur ; tree structure ; N
träd ; tree ; N
arbetskatalog ; working directory ; N
```

See the file `terms.csv` attached to the lecture
Format: <swedish> ; <english> ; <part-of-speech>

Dictionaries



```
my_dictionary = {  
  "dator": {  
    "part-of-speech": "noun",  
    "pronunciation": ["dɑ:tər", "dɑ:tur"],  
    "etymology": "From data + -or. Proposed in 1967  
                  by professor Börje Langefors, as a  
                  parallel to doktor and traktor."  
  }  
}
```

Dictionary

A python dictionary is a **data structure** which maps **keys** to **values**. Given a key, the value associated with it can be retrieved quickly.

Examples:

- key=Swedish word, value=English word
- key=word, value=the number of appearances in a text
- key=year, value=list of famous people born that year

Notes:

- there is at most one value associated with any given key
- as for lists, dictionaries do not store actual values, but only *pointers* to them.

Building a dictionary in the Python shell

```
>>> sweng = {}
>>> type(sweng)
<class 'dict'>

>>> sweng = {"attribut": "attribute",
             "klass": "class"}
>>> sweng["attribut"]
'attribute'
>>> sweng["skröp"]
KeyError: 'skröp'
>>> sweng["skröp"] = "garbage"
>>> sweng["skröp"]
'garbage'
>>> sweng
{'attribut': 'attribute',
 'klass': 'class', 'skröp': 'garbage'}
```

We can start with an empty dictionary, {}
Its Python type is **dict**

We can also list entries in the format:
<key>:<value>

A look-up has the same syntax as indexing:
<dictionary>[<key>]

If the key doesn't exist you get **KeyError**

We can add another word by using assignment
and the word as index.
Now the search succeeds.

This is how the whole dictionary looks.

Loop over the contents of a dictionary

```
>>> dict = {'attribut': 'attribute',  
            'klass': 'class',  
            'skröp': 'garbage'}
```

```
>>> for w in dict:  
...     print(w,dict[w])
```

```
attribut attribute  
klass class  
skröp garbage
```

```
>>> for w,t in dict.items():  
...     print(w,t)
```

When we use a for-loop with a dictionary, the variable gets the keys as values. The value you get by indexing.

A simpler and a more efficient way is to use method `items()`. Now `w` is the key and `t` is the value.

Default values: `get()` and `setdefault()`

```
>>> dict = {}
>>> dict["attribut"]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'attribut'
>>> dict.get("attribut",[])
[]
>>> dict
{}
>>> dict.setdefault("attribut",[])
[]

>>> dict
{'attribut': []}
```

Indexing with missing key is an error.

But we may want to consider that a missing key is equivalent to a default value (e.g. `[]`)

Method `get()` returns a default value instead.

The dictionary is still empty.

`setdefault()` acts like `get()`, but also changes the dictionary so that the key is associated to the default value.

setdefault, continued.

```
>>> dict={}
>>> mylist = dict.setdefault("attribut",[])
>>> dict
{'attribut': []}
```

```
>>> mylist.append("attribute")
```

```
>>> dict
{'attribut': ['attribute']}
```

dict is empty

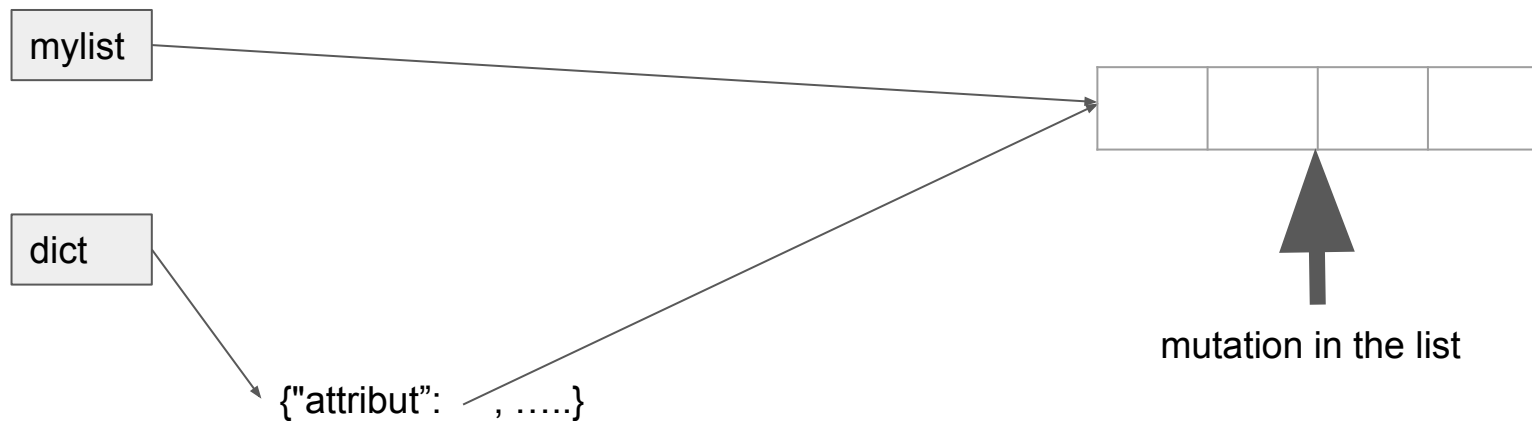
setdefault has inserted an empty list which is also references by variable mylist.

append() mutates the list itself

So, we see the change in dict as well

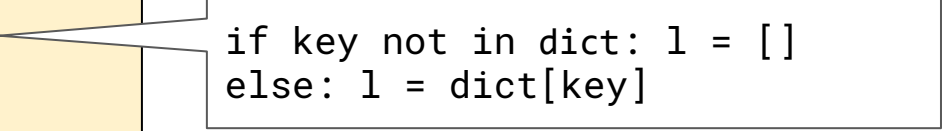
References in a dictionary

- Variables are just references
- In-place mutations can be seen from all references!



The get idiom

```
dict = {}  
l = dict.get("attribut", [])  
print(l)
```



```
if key not in dict: l = []  
else: l = dict[key]
```

The.setdefault idiom

```
def dict_list_insert(d,key,val):  
    l = d.setdefault(key,[])  
    l.append(val)
```

```
d = {}  
dict_list_insert(d,"test",1)  
print(d)           # {"test": [1]}  
dict_list_insert(d,"test",2)  
print(d)           # {"test": [1,2]}
```

Assume d a dictionary of lists

if key not in d: d[key]=[]
l=d[key]

insert val in the list (and in the dictionary,
since the dictionary points into the list).

This pattern is used often when mutating a dictionary
whose values are mutable.

Dictionaries, summary

<code>{}</code>	empty dictionary
<code>{<key>: <value>}</code>	dictionary with an entry
<code><dict>[<key>]</code>	get the value for <key>
<code><dict>[<key>] = <value></code>	change or add a value for <key> (note: dict is mutable!)
<code><dict>.get(<key>,<default>)</code>	the value for <key> if there is one, otherwise <default>
<code><dict>.setdefault(<key>,<default>)</code>	if <key> is not in <dict> , add it and set <default> as value.
<code><exp> in <dict></code>	true if <exp> is a key in <dict>
<code><exp> not in <dict></code>	true if <exp> is not a key in <dict>
<code>for <name> in <dict>: <block></code>	loop over the keys in <dict>
<code><dict>.items()</code>	returns a sequence of key value pairs in <dict>

Example: File to Dictionary

Input file

See the file `terms.csv` on Canvas

Format: `<swedish> ; <english> ; <part-of-speech>`

```
attribut ; attribute ; N
metod ; method ; N
formatera ; format ; V
formatering ; formatting ; N
muterbar ; mutable ; A
icke-muterbar ; non-mutable ; A
skräp ; garbage ; N
referera ; refer ; V
muterbarhet ; mutability ; N
effektiv ; efficient ; A
ineffektiv ; inefficient ; A
optimering ; optimization ; N
element ; element ; N
öppna ; open ; V
stänga ; close ; V
```

First version

```
def mkDict(lines):  
    dict = {}  
    for line in lines:  
        fields = line.split(';')  
  
        if len(fields) >= 2:  
            dict[fields[0].strip()] = fields[1].strip()  
    return dict  
  
def readDict(filename):  
    file = open(filename)  
    lines = file.readlines()  
    file.close()  
    return mkDict(lines)
```

To read a list of lines:

Divide each line with ';'.
Use `split()`.

Ignore lines with fewer than 2
fields.

Remove space in the beginning
and the end of every field with
`strip()`
Add **fields[0]: fields[1]**

Open the data file with `open()`
Read the lines with `readlines()`
Close the file with `close()`
Build a dictionary with `mkDict()`

Analysis of strings: split() and strip()

```
>>> s = "veni, vidi, vici"

>>> s.split()
['veni,', 'vidi,', 'vici']

>>> s.split(',')
['veni', ' vidi', ' vici']

>>> ws = s.split(',')
>>> for i in range(len(ws)): ws[i] = ws[i].strip()
...
>>> ws
['veni', 'vidi', 'vici']

>>> ' vidi, vici \t '.strip()
'vidi, vici'
```

split() without argument divides the string to a list of substrings divided by whitespaces.

split(c) with argument **c** divides the string with separator **c**. After that **c** is removed but the whitespace remains.

You need to use **strip()** to take away the space between the elements.

Now we get the list separated by comma and without whitespace.

Strip takes away all whitespace (space, tab, new line, etc. from both ends but not in the middle.

An interactive dictionary

```
import sys

def main():
    filename = sys.argv[1]

    dict = readDict(filename)

    prompt = "ange sökord+enter, sluta med enter> "

    query = input(prompt)

    while query != "":
        print(dict[query])
        query = input(prompt)
```

`python dict.py terms.csv`

The file name comes from the command line argument.

Build a dictionary from the file.

The prompt says what to do.

Read the search word.

Process the input;
If the input is empty (the user only pressed enter), the program ends.

Test the dictionary as a user

```
$ python3 dictionary.py terms.csv
```

```
ange sökord+enter, sluta med enter> skröp  
garbage
```

```
ange sökord+enter, sluta med enter> skräpsamling  
garbage collection
```

```
ange sökord+enter, sluta med enter> programspråk
```

```
Traceback (most recent call last):
```

```
  File "simple_dict_lookup.py", line 28, in <module>  
    main()
```

```
  File "simple_dict_lookup.py", line 25, in main  
    print(dict[query])
```

```
KeyError: 'programspråk'
```

What! The word is not there?

A cryptic error message, which furthermore shows implementation details to the user.

A little improvement of the dictionary with get()

```
import sys

def main():
    filename = sys.argv[1]

    dict = readDict(filename)

    prompt = "ange sökord+enter, sluta med enter> "

    query = input(prompt)

    while query != "":

        print(dict.get(query, "hittar inte"))

        query = input(prompt)
```

dict.get(<key>, <default>) returns

- the value of **<key>** if there is one
- **<default>** otherwise

Now the program doesn't crash just because the word is not in the dictionary.

A little improvement of the dictionary

```
$ python3 dictionary.py terms.csv
```

```
ange sökord+enter, sluta med enter> skräp  
garbage
```

```
ange sökord+enter, sluta med enter> programspråk  
hittar inte
```

```
ange sökord+enter, sluta med enter> programmeringsspråk  
hittar inte
```

Works like before if the word is there.

If it is not, you get a better error message, and the program doesn't crash.

But how did it happen that **terms.csv** doesn't contain any of these words?

More structure in the dictionary file

```
programspråk , programmeringsspråk ; programming language ; N  
köra , exekvera ; run , execute ; V  
programmera ; program ; V  
program ; program ; N
```

One entry may contain several words, which are separated with a comma (,) instead of semicolon (;).

These commas create lists of words, and we want to have the following from **mkDict(lines)**:

```
{'programspråk': ['programming language'],  
 'programmeringsspråk': ['programming language'],  
 'köra': ['run', 'execute'],  
 'exekvera': ['run', 'execute'],  
 'programmera': ['program'],  
 'program': ['program']}
```

Dictionary with synonyms

```
def mkDict(lines):  
  
    dict = {}  
    for line in lines:  
        fields = line.split(';')  
        if len(fields) >= 2:  
  
            for word in fields[0].split(','):   
                word = word.strip()  
  
                list = dict.setdefault(word, [])  
  
                for trans in fields[1].split(','):   
                    list.append(trans.strip())  
  
    return dict
```

(So far as before.)

Analyse every field further to comma-separated words.

If we haven't seen the word before, set an empty list in dict.

Add all translations in the list.

Bidirectional dictionary with synonyms

```
def mkDict(lines,src,tgt):  
  
    dict = {}  
    for line in lines:  
        fields = line.split(';')  
        if len(fields) >= 2:  
  
            for word in fields[src].split(','):   
                word = word.strip()  
  
                list = dict.setdefault(word,[])  
  
                for trans in fields[tgt].split(','):   
                    list.append(trans.strip())  
  
    return dict
```

src,tgt specify which field should be used as "source" (src) or "target" (tgt)

Control the direction from the command line

```
def main():
    filename = sys.argv[1]

    if len(sys.argv) > 2:
        src,tgt = sys.argv[2].split(',')
        dict = readDict(filename,int(src),int(tgt))

    else:
        dict = readDict(filename,0,1)

    ....
```

`python dict.py terms.csv 1,0`

The columns for from and to are specified in the second argument.

1,0 gives English to Swedish

If there is no second argument, the default is 0,1, i.e. from Swedish to English.

Test the new dictionary

```
$ python3 dict.py data/terms.csv
ange sökord+enter, sluta med enter> programspråk
programming language
ange sökord+enter, sluta med enter> programmeringsspråk
programming language
ange sökord+enter, sluta med enter> köra
run
execute
```

```
$ python3 dict.py data/terms.csv 1,0
ange sökord+enter, sluta med enter> program
programmera
program
ange sökord+enter, sluta med enter>
```

```
$ python3 dict.py data/terms.csv 2,0
ange sökord+enter, sluta med enter> V
programmera
installera
köra
exekvera
```

Both synonyms are searchable.

We can also translate from English to Swedish.

With field 2 we can also search all verbs, nouns, etc.

From the book

- Section 11.7